

Active Information Gathering & Scanning

Module 2 — Reconnaissance · Unit 08

Last week we *read* about the target. This week we start *talking to it* — and that changes everything legally.

Where we are in Module 2

- Unit 07 — Passive recon (OSINT): read public records.
- **Unit 08 — Active recon & scanning** ← this week: send packets to the target.
- Unit 09 — Vuln scanning & enumeration → recon mini-project.

The single change this week: **we now touch the target directly.** That changes the law.

Learning objectives (1 of 2)

By the end of this unit you can:

- **Explain** active information gathering and why it legally requires authorization.
- **Describe** host discovery and the three port states: **open**, **closed**, **filtered**.
- **Run** an nmap host-discovery (ping) scan with `-sn`.
- **Distinguish** a TCP connect scan (`-sT`) from a SYN scan (`-sS`).
- **Select** ports with `-p`, ranges, `-p-`, and `--top-ports`.

Learning objectives (2 of 2)

- **Identify** services/versions with `-sV`, attempt OS detection with `-O`, run default scripts with `-sC`.
- **Adjust** scan timing (`-T0`–`-T5`) and save results in multiple **output formats**.
- **Read and annotate** an nmap report — open ports, services, versions.
- **Grab a service banner** with netcat and explain what it reveals.

Vocabulary — concepts

Term	Meaning
Active info gathering	Directly probing a target — requires authorization.
Host discovery	Finding which hosts/IPs are alive before scanning ports.
Port	A numbered "door" where a service listens (80 = web).
Port scanning	Checking which ports are open/closed/filtered.
nmap	The standard open-source network mapper / port scanner.

Vocabulary — scan types

Term	Meaning
Ping scan (<code>-sn</code>)	Host discovery only — no ports scanned.
TCP connect (<code>-sT</code>)	Full handshake; no root needed; "noisier."
SYN scan (<code>-sS</code>)	Half-open; faster/stealthier; needs root.
Version detection (<code>-sV</code>)	Asks open ports what software/version they run.
OS detection (<code>-O</code>)	nmap's <i>guess</i> at the operating system.

Vocabulary — scripts, timing, output

Term	Meaning
Default scripts (<code>-sC</code>)	Runs nmap's safe default NSE scripts.
Timing (<code>-T0</code> - <code>-T5</code>)	How fast/aggressive the scan runs (<code>-T4</code> typical).
Banner grabbing	Reading the text a service announces.
netcat (<code>nc</code>)	Simple tool for raw network connections.
Output format	<code>-oN</code> normal, <code>-oG</code> grepable, <code>-oX</code> XML, <code>-oA</code> all.

Day 1

Active vs passive, the law, and what scanning actually does

Warm-up

Last week we **read** public records about a target. This week we **send packets to** the target.

Why might that be a bigger legal deal?

- Reading a public record touches **no one's system**.
- Sending packets is **direct interaction** — exactly the "access" computer-crime law cares about.

What active information gathering is

- **Active info gathering** = directly probing or connecting to a target's systems.
- Examples: port scanning, banner grabbing, service fingerprinting.
- It is **louder** and leaves traces (logs, IDS alerts) on the target.

Active = you reach out and touch the target. That is why it **requires authorization**.

Ports: the doors of a host

- A **port** is a numbered door where a service listens.
- Common: **80** (web), **22** (SSH), **445** (SMB), **21** (FTP).
- **Host discovery** finds which machines are alive *before* you check their ports.

Building analogy: host discovery finds which buildings are occupied; port scanning checks which doors are unlocked.

A few ports worth memorizing

Port	Service
21	FTP (file transfer)
22	SSH (remote shell)
80 / 443	HTTP / HTTPS (web)
139 / 445	SMB (Windows shares)
3306	MySQL (database)

The three port states

State	Meaning
Open	A service is listening and responding.
Closed	Reachable, but nothing is listening there.
Filtered	A firewall is blocking or dropping the probe.

Closed $\neq$ filtered. Closed = "nobody home." Filtered = "a guard won't let you knock."

Closed vs filtered — the analogy

- **Open:** you knock, someone answers.
- **Closed:** you knock, you hear *"nobody's home"* — but the door exists and replies.
- **Filtered:** a guard out front won't even let you reach the door.

The difference is *who answers*: the host (closed) or a firewall (filtered).

Verify the lab **BEFORE** scanning

```
ip a          # find your Kali IP and subnet
```

- Confirm both VMs are on **host-only** networking.
- They must reach **each other** but **not** the internet.

This is the single most important safety control of the unit.

Lab Setup + Part A — host discovery

```
sudo nmap -sn 192.168.56.0/24
```

- `-sn` = **ping scan**: find live hosts only, **no ports scanned yet**.
- Identify which IP is your target (the isolated Metasploitable VM).

Write your **scope statement** first: *"I am authorized to scan only the isolated lab target."*

Check your understanding

A real third party's server crashes for **unrelated** reasons one minute after your unauthorized scan. Nothing of yours "broke" anything.

Were you in the clear? **Why or why not?**

Think before the next slide.

Answer

No. Sending packets without authorization can be a **crime by itself** — or evidence of attempted intrusion — *regardless* of what broke.

- The unauthorized **access attempt** is the violation.
- There is no "but nothing broke" or "just curious" defense.

Exit ticket — Day 1

Why does scanning a real third-party server without permission risk being a **crime**, even if nothing breaks?

Day 2

Host discovery and port scanning (`-sn`, `-sT` vs `-sS`, `-p`)

Warm-up

What's the difference between finding which **houses are occupied** and checking which **doors are unlocked**?

- Occupied houses → **host discovery** (`-sn`)
- Unlocked doors → **port scanning** (`-sT` , `-sS`)

First you find what's alive; then you probe it.

The TCP 3-way handshake

Before any TCP scan makes sense, know the handshake:

1. Client → **SYN** ("let's talk")
2. Server → **SYN/ACK** ("sure, go ahead")
3. Client → **ACK** ("connected")

Scans differ in **how much** of this they complete.

TCP connect scan (-sT)

```
nmap -sT --top-ports 20 <target-ip>
```

- Completes the **full** 3-way handshake (SYN → SYN/ACK → ACK).
- **Works without root** — anyone can run it.
- "Noisier": the full connection is more likely to be **logged**.

Use **-sT** when you don't have root privileges.

SYN scan (`-sS`)

```
sudo nmap -sS <target-ip>
```

- "**Half-open**": SYN, gets SYN/ACK, then sends **RST** — never finishes.
- **Faster and stealthier** than `-sT`.
- **Needs root** (`sudo`).

Use `-sS` when you have root and want speed/stealth.

-sT vs **-sS** side by side

	-sT connect	-sS SYN
Handshake	Full 3-way	Half-open (SYN, then RST)
Root needed?	No	Yes
Speed	Slower	Faster
Profile	Noisier, logged	Stealthier

Same goal (find open ports), different trade-offs.

Selecting ports — flag by flag

```
nmap -p 22,80,443 <target-ip>      # specific ports
nmap -p 1-1000 <target-ip>         # a range
nmap --top-ports 20 <target-ip>    # the most common ports
```

- `-p 22,80,443` → just those three.
- `-p 1-1000` → every port in the range.
- `--top-ports 20` → nmap's 20 most-common ports.

Scanning every port: `-p-`

```
sudo nmap -p- <target-ip>          # ALL 65,535 ports
```

- `-p-` is shorthand for `-p 1-65535`.
- **Thorough** — catches services hiding on odd ports.
- **Slow** — use it when the wait is worth it.

Start small, broaden as you learn the target.

Lab Part B — progressive port scans

```
nmap -sT --top-ports 20 <target-ip>  
sudo nmap -sS <target-ip>  
nmap -p 21,22,80,445 <target-ip>  
sudo nmap -p- <target-ip>
```

Record for each scan: ports and their **state**. In your own words, explain `-sT` vs `-sS`.

Check your understanding

You run `sudo nmap -sS <target>` but every single port comes back as the same state and the output looks empty/weird.

What's the most likely cause?

Think before the next slide.

Answer

You probably **dropped** `sudo` (or it failed).

- `-sS` (SYN) and `-O` (OS) need **root** to craft raw packets.
- Without root, nmap silently falls back or returns junk.

Fix: rerun with `sudo`, or use `-sT` if you can't get root.

Exit ticket — Day 2

When would you choose `-sT` over `-sS`?

Day 3

Fingerprinting: service/version (`-sV`), OS (`-O`), default scripts (`-sC`)

Warm-up

An open port tells you a **door exists**. How would you learn **what's behind** the door?

You ask it. nmap can:

- Ask the service its **name and version** (`-sV`)
- Guess the **operating system** (`-O`)
- Run small scripts for **extra detail** (`-sC`)

Service/version detection (`-sV`)

```
nmap -sV <target-ip>
```

- Connects to each open port and asks: *what software, what version?*
- Output like `21/tcp open ftp vsftpd 2.3.4`.

The **version** is the prize: it maps to known vulnerabilities. (More in Unit 09.)

OS detection (-0)

```
sudo nmap -0 <target-ip>
```

- nmap fingerprints how the host replies and **guesses** the OS.
- Example: "Linux 2.6.x" — a *best guess*, not fact.
- **Needs root.**

Treat **-0** as a hint, never as proof.

Default scripts (`-sC`)

```
nmap -sC <target-ip>
```

- Runs nmap's **safe default NSE scripts** to gather extra detail.
- Can reveal SSL info, supported auth methods, or directory listings.

A free first pass of automated enumeration — bundled into the scan.

Combining it all

```
sudo nmap -sS -sV -sC -O -p- <target-ip>
```

Flag	Does
-sS	SYN scan (stealthy, needs root)
-sV	service & version detection
-sC	default scripts
-O	OS guess
-p-	all 65,535 ports

37

Star every version number

When you read the `-sV` output, **mark each version**.

- `vsftpd 2.3.4`, `OpenSSH 4.7p1`, `Apache 2.2.8` ...
- Each one is a **lead** you'll research in Unit 09.

Versions are the bridge from "what's there" to "what's weak."

Lab Part C — fingerprint & annotate

```
sudo nmap -sS -sV -sC -O -p- <target-ip>
```

Record for each open port:

- the **service** and **version** nmap reports
 - the **OS guess** (note: a guess)
 - any interesting `-sC` script output
- ★ the version numbers — your leads for Unit 09.

Check your understanding

-0 reports "Linux 2.6.x." A classmate writes in their report: *"The target runs Linux 2.6."*

What's wrong with that sentence?

Think before the next slide.

Answer

-0 is a **guess**, not a fact.

- Better: *"nmap's OS detection **estimates** Linux 2.6.x."*
- Phrase it as an inference, then confirm with other evidence (banners, behavior).

Professionals never present a guess as proof.

Exit ticket — Day 3

Why is the **version** number of a service so valuable to both attackers and defenders?

Day 4

Timing, output formats, and reading scan results

Warm-up

Why might a scanner run **slowly** on purpose?

- A fast, aggressive scan is **easy to detect** — it spikes traffic and trips alarms.
- A slow scan blends into normal traffic — **stealthier**, but takes much longer.

Speed vs stealth is a trade-off you control with **timing templates**.

Timing templates (-T0 - -T5)

Template	Behavior
-T0 / -T1	Very slow — evades detection
-T3	Default
-T4	Faster — typical in the lab
-T5	Fastest — can miss results

Timing in practice

```
nmap -sT -T1 --top-ports 10 <target-ip> # slow / stealthy  
nmap -sT -T4 --top-ports 10 <target-ip> # faster
```

- `-T1` drags out timing to dodge alarms.
- `-T4` speeds it up for a cooperative lab network.

Higher T = faster + louder + more likely to miss things.

Output formats

```
sudo nmap -sS -sV -T4 -oA scan_target <target-ip>
```

Flag	File	Best for
<code>-oN</code>	<code>.nmap</code>	Humans to read
<code>-oG</code>	<code>.gnmap</code>	<code>grep</code> / scripting
<code>-oX</code>	<code>.xml</code>	Feeding other tools
<code>-oA</code>	all three	Saving everything

Professionals save scan output as **evidence** for the report.

Why save scans as evidence

- A report claim needs **proof** — the raw scan is your proof.
- Saved output makes results **reproducible** by a reviewer.
- `-oA` captures all three formats in one shot — no redoing the scan.

If it isn't saved, it didn't happen (for reporting purposes).

Reading an nmap report

A typical line:

```
21/tcp  open  ftp      vsftpd 2.3.4
```

Field	Meaning
21/tcp	Port number / protocol
open	Port state
ftp	Service
vsftpd 2.3.4	Software + version

49

Lab Part D — timing & output

```
sudo nmap -sS -sV -T4 -oA scan_target <target-ip>  
grep open scan_target.gnmap      # quick open-port list
```

Record: cite the saved files; note which format you'd **grep** vs **hand to a tool**; one sentence on `-T1` vs `-T4`.

Check your understanding

You need a quick list of just the open ports from a saved scan, for a Bash script.

Which saved file do you reach for, and why?

Think before the next slide.

Answer

The **grepable** file: `scan_target.gnmap` (from `-oG`).

- One host per line, easy for `grep` / `awk`.
- The `.xml` file is for **feeding other tools**, not quick grepping.
- The `.nmap` file is for **humans** to read.

Exit ticket — Day 4

Which output format would you **grep** for `open 80/tcp`, and which would you **hand to another tool**?

Day 5

Banner grabbing with netcat + annotated results
writeup

Warm-up

nmap **guessed** the service with `-sV`. How could you **confirm** it by hand?

You connect to the port yourself and read what it tells you — the **banner**.

Banner grabbing with netcat

```
nc <target-ip> 21      # FTP banner  
nc <target-ip> 22      # SSH banner
```

- A **banner** is the text a service announces — often its name and version.
- netcat reads/writes raw connections — the simplest way to talk to a service by hand.

Grabbing an HTTP banner

```
nc <target-ip> 80  
HEAD / HTTP/1.0  
(press Enter twice)
```

- HTTP won't talk until **you** speak first.
- The reply's `Server:` header names the web software.

Different services, different etiquette — but all announce themselves.

Comparing banner to `-sV`

Port	Banner (netcat)	nmap <code>-sV</code>
21 (FTP)	220 (vsFTPD 2.3.4)	vsftpd 2.3.4
22 (SSH)	SSH-2.0-OpenSSH_4.7p1	OpenSSH 4.7p1

They should **agree**. `vsftpd 2.3.4` is a famously backdoored version — **note it as a Unit 09 lead; do NOT exploit it here.**

Why grab a banner by hand at all?

- It **confirms** nmap's guess with a second source.
- Some services reveal **more** in the raw banner than `-sV` summarizes.
- It builds intuition for how services actually talk.

Two sources agreeing turns a guess into a confident finding.

Lab Part E — banners + annotated results

```
nc <target-ip> 21  
nc <target-ip> 22
```

Record: the raw banner; compare to nmap's `-sV`. Did they agree?

Then assemble your **annotated scan results**: paste the fingerprinting output and annotate each open port with service, version, and why it matters.

Ethics & Authorization

This unit sends packets to a target. That is direct-access territory.

Port-scanning a system you don't own or aren't authorized to test **can itself be a crime** — or evidence of attempted intrusion — even if nothing breaks.

No "just curious" exception

- The **same nmap command** is legal against our lab VM and a **crime** against a stranger's server.
- The only difference is **written authorization + a defined scope.**
- Our scope: the isolated **Metasploitable 2** VM and approved **TryHackMe** rooms. **Nothing else.**

nmap ships on every Kali box — **never** point it at the school network or the internet.

Discussion

Knocking on every door in a neighborhood to see which are unlocked isn't "breaking in."

But is it **harmless**? How is **port scanning** similar? Why does the law still care — and why does **authorization** change the answer?

Recap

- Active recon **touches the target** — authorization is mandatory.
- Host discovery first (`-sn`), then ports, then fingerprinting.
- Port states: open / closed / **filtered** (firewall) $\neq$ closed.
- `-sT` (no root, noisy) vs `-sS` (root, stealthy). `-0` is a *guess*.
- **Versions are the gold** — they become Unit 09's vuln leads.
- Save output as evidence. Stay on the lab VM, every day.

Key vocabulary — quick review (1 of 2)

Term	Meaning
<code>-sn</code>	Ping scan (host discovery only)
<code>-sT</code> / <code>-sS</code>	TCP connect / SYN (half-open) scan
<code>-sV</code> / <code>-O</code> / <code>-sC</code>	Version / OS / default-script detection
<code>-p-</code> / <code>--top-ports</code>	All ports / most common ports

Key vocabulary — quick review (2 of 2)

Term	Meaning
Port state	open / closed / filtered
Banner grabbing	Reading the text a service announces
<code>-oN</code> / <code>-oG</code>	Normal / grepable output
<code>-oX</code> / <code>-oA</code>	XML / all formats output

Exit ticket — Day 5 + homework

Exit ticket: submit annotated scan results; one sentence on the authorization line for active scanning.

Homework:

- Complete the nmap flag cheat sheet from memory.
- ½-page: "Why is the exact same nmap command legal against our lab VM but a potential crime against a stranger's server?" Use **authorization** and **scope**.

Next up — Unit 09: Vulnerability Scanning & Enumeration + the Module 2 recon mini-project.